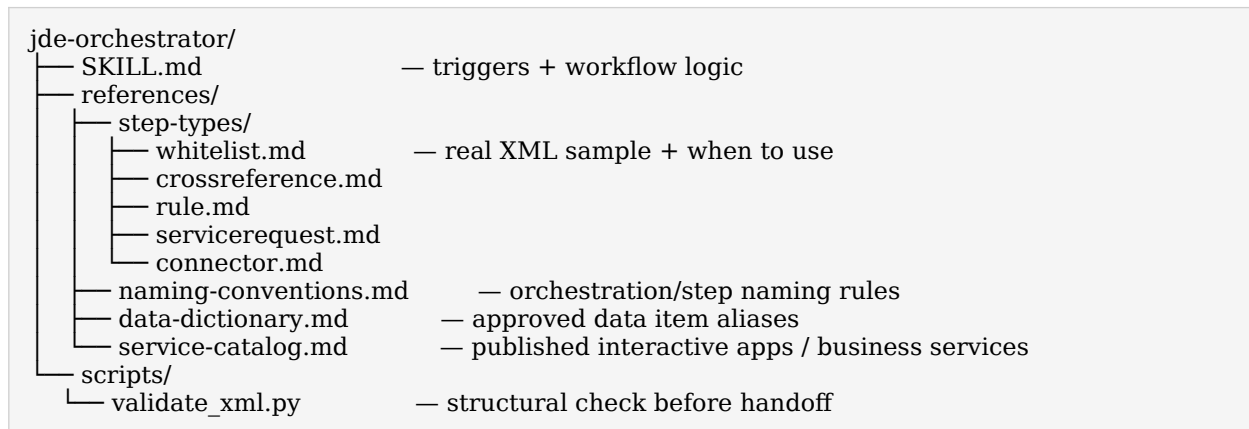


Building a Claude Skill for JDE Orchestration

Skill structure and knowledge context for generating orchestration XML

Building this as an actual Claude skill (not just a one-off prompt) gives you the “progressive disclosure” structure — the skill loads a short description always, the fuller instructions when it triggers, and the heavy reference material only when needed. Here's how to lay it out for JDE orchestration generation specifically.

Skill Anatomy for This Use Case



What Goes in SKILL.md (the “prompt” part)

Keep it under roughly 500 lines. It should cover the workflow, not the syntax details:

1. **Description (frontmatter)** — this is what triggers the skill, so make it specific: “Use whenever the user describes a business requirement that needs to become a JDE Orchestrator orchestration, or mentions orchestration XML, WhiteList/CrossReference/Rule/ServiceRequest steps, or Orchestrator Studio import.”
2. **Workflow steps** — parse the requirement into: inputs needed, sequence of steps, conditional branches, which existing published services get called. Point to the right references/step-types/*.md file for each step type involved rather than embedding all the XML syntax inline.
3. **Hard rules** — the single most important rule: never invent a data item alias or service name — flag it as “NEEDS VERIFICATION” if not certain it exists in the catalog. A hallucinated service reference will silently fail import, or worse, succeed against the wrong object.
4. **Output format** — always produce a complete, importable XML file matching the reference schema, plus a short changelog of assumptions made.

Building the Knowledge Context

This is really the RAG-style ground-truth layer, and it's the part that actually determines whether generated XML imports cleanly:

Real exported samples, one per step type

Export 4–5 orchestrations from your Studio, split by step type into individual reference files. This is what stops Claude from generating a plausible-looking but wrong tag structure.

Data dictionary excerpt

A list of the data item aliases your org actually uses (not the full JDE dictionary — just what's relevant to sensor/equipment/whatever domains you're orchestrating for). Without this, Claude will invent alias names that look right but don't exist.

Service / interactive-app catalog

The published business services and interactive application versions available for ServiceRequest and Connector steps — names, expected inputs. This is the highest-risk gap, since a wrong service reference is a runtime failure, not a validation failure.

Naming conventions doc

Your org's pattern for orchestration names, step names, versioning — keeps generated output consistent with what's already in Studio.

A few worked examples

2–3 requirement → final-XML pairs, ideally ones a developer already reviewed and imported successfully. Few-shot examples like this do more for output quality than lengthy prose instructions.

Validation Script

If Oracle publishes an XSD for the orchestration schema (worth checking your Tools Release docs), wrapping it in a small Python script under scripts/ lets the skill self-check structural validity before handing the XML to a human — catching malformed tags before someone wastes a review cycle on it.

Next Step

Once real samples are available, the actual SKILL.md and reference files can be drafted, then run against a few test requirements so the output can be sanity-checked before it ever touches Orchestrator Studio — the standard draft → test on realistic prompts → review → refine loop.